Раздел 3.2.

Понимание внутреннего устройства моделей

3.2.1. Оцифровка языка: токенизатор

3.2.2. Архитектура Transformer

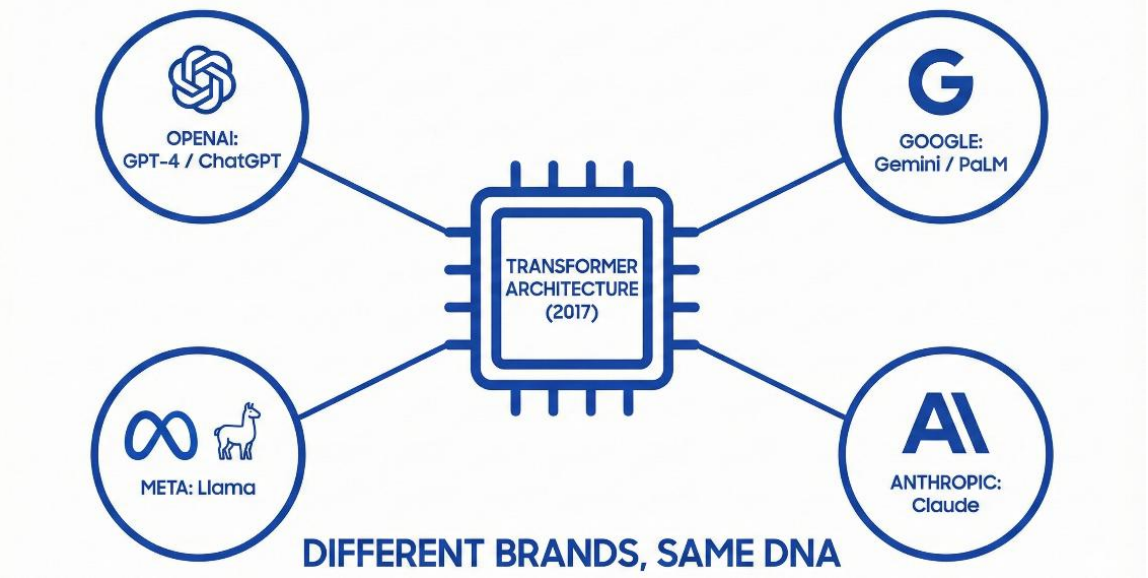
3.2.3. Предварительное обучение и данные

3.2.4. Понимание процесса вывода (инференса)

Универсальность: Transformer

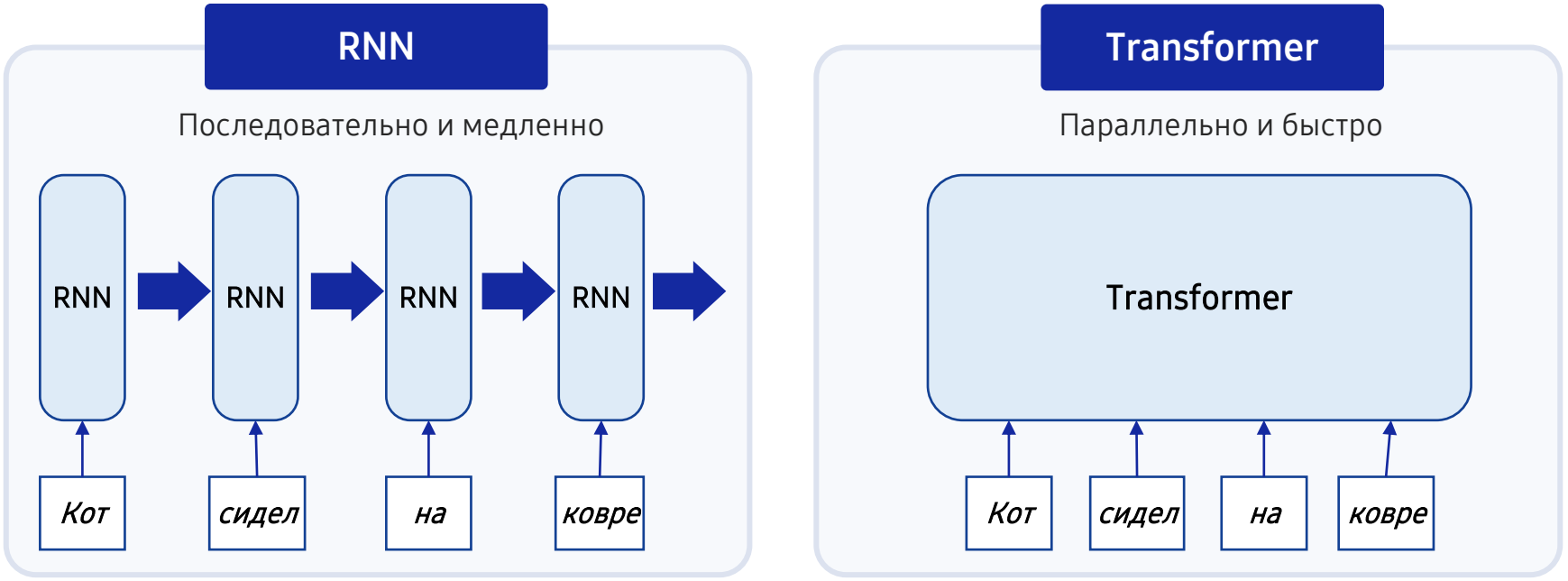
- Независимо от бренда, «ДНК» всех современных базовых моделей — это архитектура Трансформер (Transformer)
  - Универсальный движок: общая архитектура, лежащая в основе современных систем ИИ
  - Общая основа: используется в GPT, Gemini, Llama, Claude и др.

Transformer: универсальный движок в ведущих LLM



От последовательного к параллельному

- Изменение парадигмы в обработке данных
  - RNN (рекуррентные нейронные сети) работают последовательно, это медленно и ограничивает окно контекста
  - Трансформер обрабатывает токены параллельно, что обеспечивает быстрое и эффективное понимание контекста



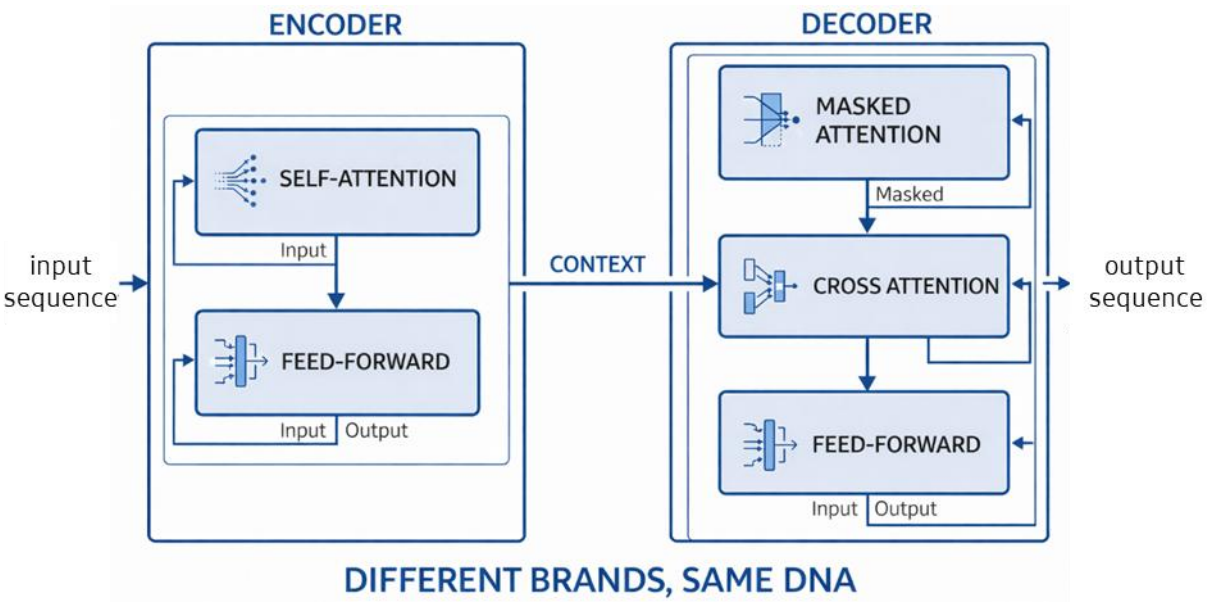
Эволюция обработки: RNN (последовательная) против Transformer (параллельная)

Анатомия Transformer

- Схема современного ИИ: двойная архитектура, «читатель» (энкодер) и «писатель» (декодер)

- Энкодер (Encoder)
  - Понимание и извлечение признаков
  - Применение: входные данных для перевода, поиск (BERT)

- Декодер (Decoder)
  - Генерация следующего токена.
  - Применение: творческое письмо, чат-боты.

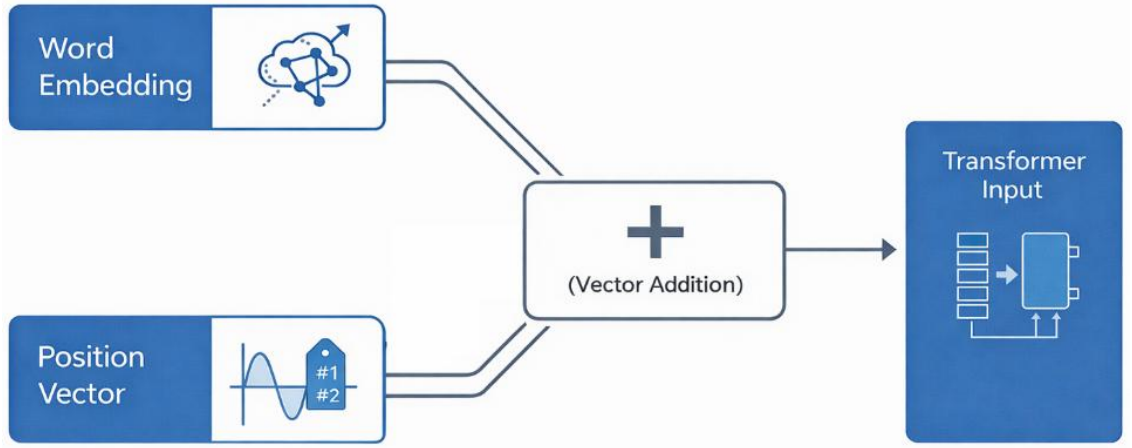


Внутри архитектуры Transformer

Обучение порядку: Позиционное кодирование

- Решение проблемы «слепого пятна» параллельной обработки: внедрение информации о последовательности путем добавления математических «позиционных тегов» (Position Tags) к векторным представлениям слов (Word Embeddings)
- Проблема: при параллельной обработке теряется информация о последовательности
  - «Человек кусает собаку» против «Собака кусает человека».
- Решение: добавление «позиционных тегов» к каждому векторному представлению слова

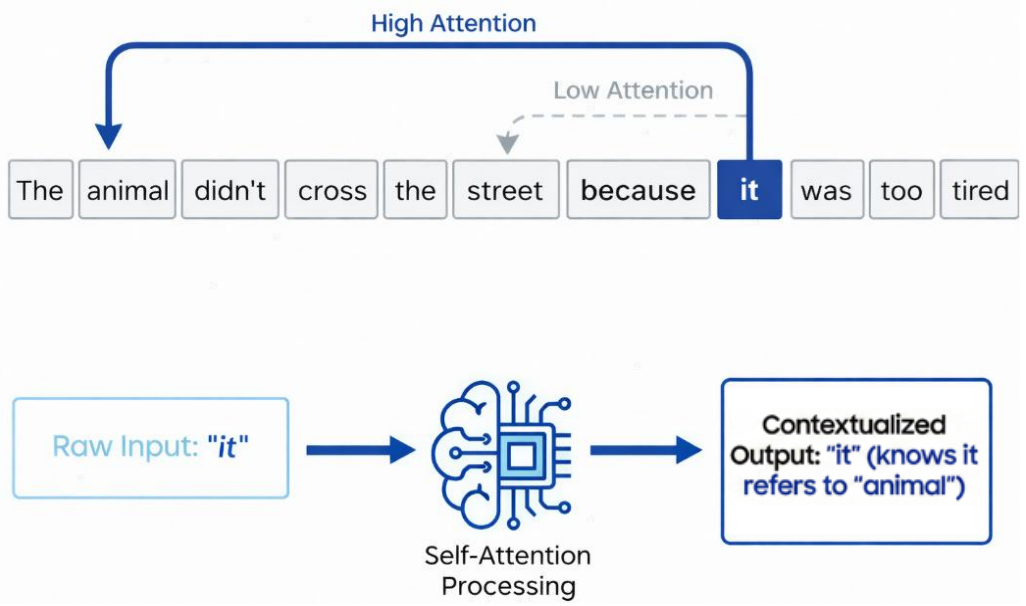
Позиционное кодирование:
добавление порядка к смыслу



Чтение контекста: механизм Self-Attention

- Устранение неоднозначности путем вычисления «весов релевантности» по всему предложению для понимания взаимосвязей между словами.
- Концепция: вычисление «веса релевантности» между словами.
- Пример: «Животное не перешло улицу, потому что оно было слишком уставшим».
  - Связи: «оно» <-> «животное» (высокое внимание)
  - Отсутствие связи: «оно» <-> «улица» (низкое внимание)

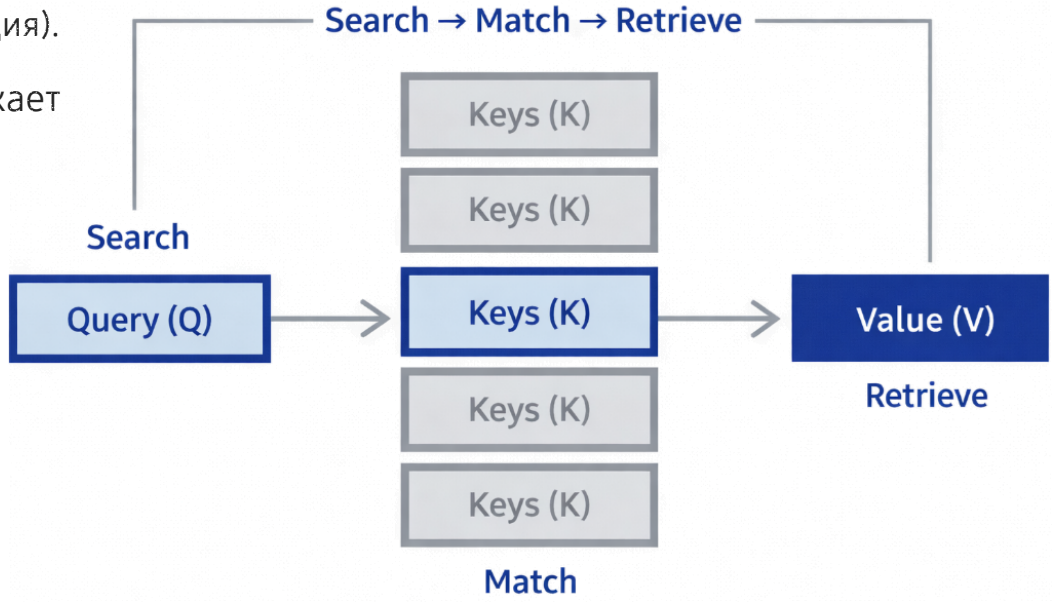
Self-Attention: вычисление релевантности для устранения неоднозначности



Query (запрос), Key (ключ), Value (значение)

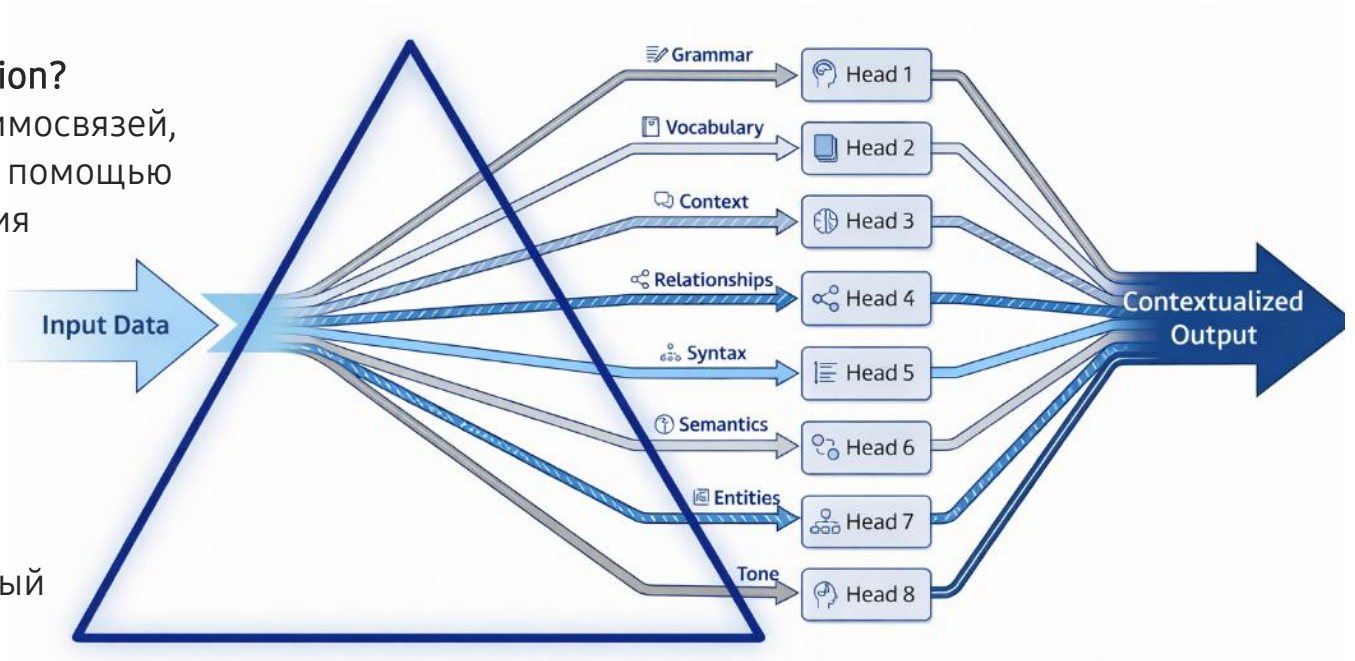
- Механизм извлечения, работающий как «система поиска в библиотеке» для нахождения наилучшего соответствия между словами.
- Аналогия (поиск в библиотеке):
  - Query (Q): то, что я хочу найти (вопрос).
  - Key (K): ярлык на корешке книги (индекс).
  - Value (V): содержание внутри книги (информация).
- Математика: функция Attention(Q, K, V) извлекает наилучшее соответствие.

Механизм Q, K, V: поиск, сопоставление и извлечение



Многоголовое внимание (Multi-Head Attention)

- **Разделяет один входной сигнал на несколько голов внимания** для параллельного захвата различных аспектов смысла
- **Зачем нужно Multi-Head Attention?**
  Язык содержит множество взаимосвязей, которые невозможно уловить с помощью одного представления внимания
- **Как это работает**
  Каждая голова фокусируется на своем аспекте (например, синтаксис, семантика, контекст), а их результаты объединяются в единый контекстуализированный выходной сигнал



Multi-Head Attention – механизм призмы:
1 входной сигнал разделяется на несколько голов

Полносвязные сети (FFN*): блок обработки

- Слои полносвязных сетей преобразуют полученную информацию в осмысленное понимание.
  - После того как механизм внимания выбрал, на чем сосредоточиться, этот слой решает, как это интерпретировать
- Зачем это нужно?
  - Реальные языковые паттерны не являются простыми или линейными
  - Смысл часто зависит от комбинаций, контекста и тонких нюансов
- Ключевая идея
  - Полносвязные сети включают механизм, который позволяет модели выйти за рамки простых линейных вычислений, позволяя ей представлять сложные и нелинейные языковые паттерны

*) Feed-Forward Networks

Где хранятся знания?

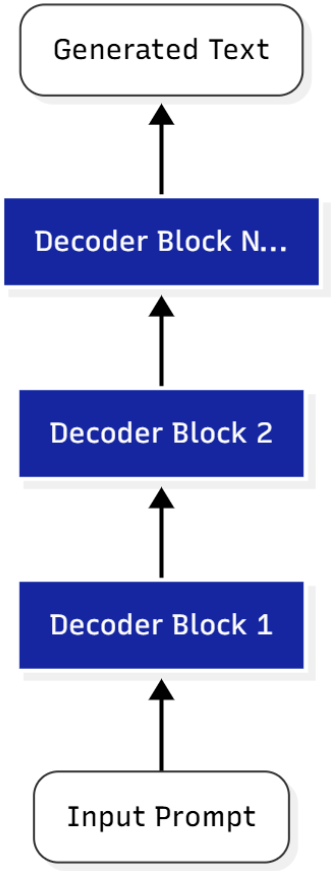
- Хранение огромных объемов знаний обеспечивается силой связей (параметров), выступающих в роли «долговременной памяти» модели
- Концепция: FFN выступает в роли «долговременной памяти» модели
- Параметры (веса):
  - Факты (история, наука) и навыки (программирование) хранятся в весах связей FFN
- Масштаб: Больше параметров ≈ Больше знаний

Устойчивость глубоких трансформеров: остаточные связи и нормализация

- Обеспечение хорошего обучения в глубоких сетях
  - «Магистрали» для беспрепятственного потока данных и «стандартизированные тесты» для согласованных диапазонов данных
  - Проблема: Информация и градиенты деградируют по мере увеличения глубины сетей (затухающий градиент)
- Решение
  - ① Остаточные связи (Residual Connections)
    - Обеспечивают кратчайшие пути, позволяющие информации пропускать слои
    - Помогают сохранить поток градиента через глубокие слои модели
  - ② Нормализация слоев (Layer Normalization)
    - Стандартизирует активации на каждом слое
    - Улучшает стабильность обучения и сходимость
- Результат
  - Обеспечивает стабильное обучение очень глубоких моделей на базе архитектуры Transformer.

Современные LLM – это только декодер

- Приоритет творчества и масштабируемости: оптимизация архитектуры до «только декодера» для эффективного предсказания следующего токена
- Сдвиг: GPT, Llama и Gemini используют только декодер
- Причина:
  - Эффективность прогнозирования следующего токена
  - Масштабируемость (легче добавлять больше слоев)
- Цель: максимизировать творчество и генерацию



Расцвет архитектуры «только декодер» (в стиле GPT)

Пример: Samsung Note Assist

- Мгновенное преобразование хаотичных протоколов встреч в упорядоченную структуру с помощью Multi-Head Attention
- Задача: суммаризация беспорядочных протоколов встреч
- Многоголовое внимание в действии:
  - Фиксирует спикеров (Кто) / ключевую повестку (Что) / задачи (Что сделать)
- Результат: структурированный и точный отчет

Реальное применение:
Samsung Note Assist

